Secure SQL Agent: Invariant-Bounded Conversational Access to Enterprise Data

Brad Long

Independent Research*
Brisbane, Australia

7 October 2026

Abstract

Secure SQL Agent is a working prototype showing that a conversational agent can answer natural-language questions over an enterprise relational database without the language model holding any authority over what data it may read. A local model (qwen3-coder, served by Ollama) interprets each question and proposes a single SQL statement against a small, security-filtered slice of the schema. Deterministic code then parses that proposal into an abstract syntax tree, checks it for structural safety and per-table and per-column authorisation, rewrites it to strip restricted columns and impose a row limit, re-validates it, and runs it through one controlled, read-only execution path. Results are filtered a second time before the model sees them to compose an answer.

The design is a concrete instance of the Invariant-Bounded Agent Alignment Model (IBAAM): read-only access and authorised-data-only are invariants held above the planner, and the set of deterministic checks enforcing them is called the Agent Behavioural Controls (ABCs). A Northwind walkthrough shows the agent following a multi-turn conversation to the right customer record, yet declining to reveal a restricted phone number. The paper describes the architecture, schema-grounding method and controls, maps them to IBAAM, and sets out the limitations that separate the prototype from an enterprise deployment.

Keywords: natural-language interfaces to databases, text-to-SQL, LLM agents, deterministic authorisation, runtime governance, invariant enforcement, IBAAM

1. Introduction

The objective of Secure SQL Agent is to make enterprise data accessible through natural language without compromising security, accuracy or control. Large language models are now good enough at writing SQL that generating a plausible query is the easy part. The hard part is everything around it: deciding which data the model may see, which statements may run, how much they may return, and what reaches the user.

* This work was conducted independently and does not represent the views of any employer. Author contact: bradlong.phd@gmail.com.

Most text-to-SQL agents pass the user's question and some description of the database to a model, then execute whatever SQL comes back. The model therefore decides which tables to touch, how much to return and when to stop. Security, if present, usually lives in the system prompt ("only write SELECT statements", "never return phone numbers"). A prompt is a request to the model, not a constraint on the system. A loosely worded question such as "clean up test accounts" can become a `DELETE` against production, and a prompt-level rule can be ignored, misread or talked around.

Secure SQL Agent starts from the opposite premise: **the LLM interprets intent; deterministic code enforces security.** The model's output is treated as a proposal, never a command. It never holds a database connection or credential, and nothing it produces can widen what the system is allowed to do. This is the central idea of the Invariant-Bounded Agent Alignment Model (IBAAM) (Long, 2026a): invariants sit above the planner and tool-execution layers, so an agent can propose an action but cannot override the constraints governing it.

This paper makes four contributions:

1. A reference architecture for a conversational SQL agent in which the model is confined to interpretation (SQL proposal and result explanation) and all authority sits in deterministic code.
2. A dynamic, security-filtered schema-grounding method that supplies the model only the tables, columns and foreign-key relationships relevant to the current question, so the approach can scale to schemas with thousands of tables.
3. A named set of Agent Behavioural Controls (ABCs): SQL validation and authorisation, SQL rewriting and safeguards, controlled execution, and result security. These form the agent's behavioural boundary.
4. A mapping of the implementation onto IBAAM's invariant, adaptive, control, execution and observability layers, giving the governance series a concrete, runnable example in an enterprise-data setting.

The prototype was implemented on PostgreSQL, with a public demonstration at sqlagent.net (Long, 2026g). It runs against the Northwind sample database, and the code excerpts and parameters in this paper refer to that implementation.

2. Background and related work

2.1 Natural-language interfaces to databases

Translating questions into SQL has a long history, and LLMs have moved it from research benchmarks such as Spider (Yu et al., 2018) and BIRD (Li et al., 2023) into everyday tools. Work in this area has focused mainly on accuracy: picking the right tables, writing correct joins, and handling ambiguous wording. Two practical problems receive less attention. First, real enterprise schemas are large, and passing the whole schema in every prompt is costly and

noisy. Second, accuracy says nothing about whether a given user should see a given answer. A perfectly correct query that returns a restricted column is a security failure.

2.2 Prompt-level guardrails and their limits

Agent frameworks such as ReAct (Yao et al., 2023) and Toolformer (Schick et al., 2023) let models reason and call tools, but they do not enforce system-level constraints: behaviour remains prompt-dependent and probabilistic (Long, 2026a). A common response is to put rules into the system prompt. Prompt rules are useful for steering the model, but they cannot be a security boundary, because the component they constrain is the same component that interprets them. Runtime governance frameworks improve on this by adding external policy enforcement, though they often mediate around an agent rather than define the agent's execution semantics (Long, 2026a).

2.3 The IBAAM governance series

IBAAM treats an agent as a system rather than an entity and separates what is allowed to change from what must remain stable. Its reference architecture has four layers: an execution layer (LLM plus tools), an adaptive layer (prompts, memory, strategy), a control layer (policies and invariants) and an observability layer (logs, traces, replay). The control layer "must not be freely modifiable by the agent itself. If the system can change its own constraints, then those constraints lose meaning" (Long, 2026a). Access control is named explicitly as a safety invariant: the agent may only access resources explicitly authorised by its execution environment.

Later papers add drift measurement (Long, 2026b; Perrier & Bennett, 2026), hierarchical and context-aware conflict resolution between invariants (Long, 2026c; 2026d) and a proposed conscience layer (Long, 2026f); InvariantAgent2 implements these ideas in a general agent runtime (Long, 2026e).

Secure SQL Agent sits at the start of this arc. It does not need hierarchy, drift metrics or conscience, because its invariants do not conflict and its behaviour is a single governed transition per question. What it adds is an applied demonstration in a domain where the stakes are concrete (i.e. enterprise data) and where IBAAM's control layer can be implemented almost entirely with established, verifiable techniques: parsing, allow-lists, query rewriting and database-enforced read-only execution.

3. System architecture

Every question follows one fixed path in which the model is consulted twice, a) to propose SQL and, b) to explain results, and every step between those two calls is deterministic code. Figure 1 shows the path. Table 1, shown after the figure, lists the components of the architecture.

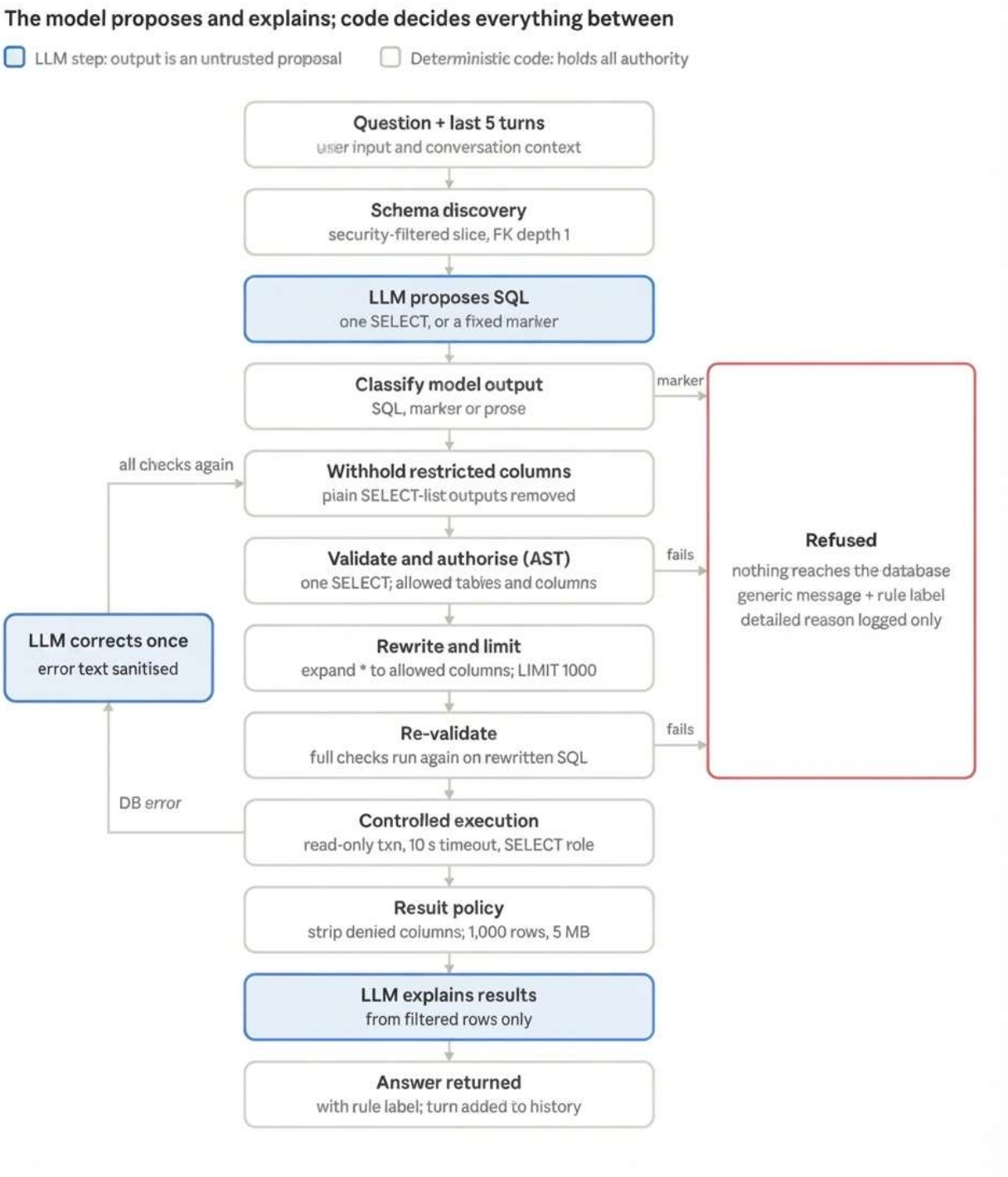


Figure 1. The Secure SQL Agent request path. Shaded steps are the only places the model acts; every other step is a deterministic control, and a failure at any check ends the request before the database is touched. A database error allows one model correction, which re-enters the checks.

3.1 Components

Component	Module	Responsibility	Trusted?
Security context	`controls/security_policy.py`	Builds the caller's security context (roles, authorised tables, denied columns) from the security policy	Yes — supplied by the host, never by the model
Schema catalog	`adaptive/schema_catalog.py`, `adaptive/semantic_model.py`	Loads tables, columns and foreign keys at startup, filtered by the security context; selects a relevant slice per question	Yes
LLM interface	`adaptive/llm.py`	Prompts for SQL generation, one-shot correction and result explanation; cleans and classifies model output	No — output is a proposal
Agent controls	`controls/sql_policy.py`, `controls/result_policy.py`	AST parsing, structural safety, authorisation, column stripping, `SELECT *` rewriting, row limits, result policy, block rules	Yes
Execution gate	`execution/sql_execution.py`	Final structural check, read-only transaction, statement timeout, least-privilege role	Yes
Diagnostics	`execution/sql_diagnostics.py`	Classifies database errors to guide one correction attempt, without bypassing authorisation	Yes
Orchestrator	`agent.py`	`run_agent()`: runs the pipeline and returns a structured result (answer, blocked, reason, rule, SQL)	Yes
Public API	`api.py`	Rate limits, concurrency gate, timeouts, input-length cap, audit log	Yes

Table 1. Components of Secure SQL Agent. "Trusted" means the component's output can change what the system is allowed to do. Apart from api.py, module paths are relative to the sqlagent package, whose adaptive, controls and execution packages follow IBAAM's layers.

3.2 The request path

1. **Question and conversation context.** The question is paired with the last five turns of conversation (Section 4.4).
2. **Schema discovery.** A security-filtered slice of the schema is selected for the question (Section 4).
3. **SQL proposal.** The model returns one SELECT statement or one of three fixed markers.
4. **Output classification.** Code decides whether the reply is SQL, a marker or prose; markers can only lead to a refusal.
5. **Column withholding.** Restricted columns are removed from the SELECT list (Section 5.2).
6. **Validation and authorisation.** The SQL is parsed and checked for structure, tables and columns (Section 5.1).
7. **Rewriting and safeguards.** Wildcards are expanded and a row limit is added (Section 5.2).
8. **Re-validation.** The rewritten SQL passes the full validation again (Section 5.2).
9. **Controlled execution.** The query runs read-only under a timeout, as a SELECT-only role (Section 5.3).
10. **Result security.** Rows are filtered and capped again (Section 5.4).
11. **Explanation.** The model writes the answer from the filtered rows.
12. **Response and history.** The answer and any rule label are returned, and the turn is added to history.

If execution fails, the error is diagnosed, schema-revealing `DETAIL` and `HINT` text is removed, and the model gets exactly one correction attempt. The corrected SQL goes through steps 5–10 again. If the correction is identical to the failed query, the agent stops.

3.3 Security policy and semantic model

Two kinds of YAML configuration drive the agent, and they are deliberately kept apart. The security policy states who may see what: roles, allowed tables and denied columns, with identity-provider groups mapped to roles. It denies by default and fails closed: a missing file, invalid YAML, an unknown role or a denied column on a table the role is not granted stops the server from starting. It is owned by security or IT.

The semantic model states what the data means: which tables to discover, business labels for cryptic names, aliases, relationships the database does not declare, terms that pick rows ("shipped late" means `shipped_date > required_date`) and the Domains of Discourse used for routing (Section 4.3). It is owned by the people who know each business area and is applied only after security filtering, so anything naming a table or column the role cannot see is skipped. The semantic model can therefore change freely: it shapes what the model is shown but can never widen access.

4. Schema grounding

The model never receives the whole database schema; it receives a small, security-filtered slice chosen for the current question. This keeps the prompt focused, reduces token cost, and lets

the design scale to databases with thousands of tables, most of which are irrelevant to any given question. Grounding is split into three concerns: **discovery** (what structure exists), **selection** (which part this question needs) and **generation** (the model writing SQL from that part).

4.1 Discovery: a security-filtered catalog

At startup the catalog reads every table and column from the database's information schema, and every foreign key from the system catalog. Filtering happens at load time. A table not in the caller's security context is never added. A denied column is never added. A foreign key is kept only when both of its tables are authorised.

This matters for security as well as focus. The model cannot reference what it has never been shown, so restricted structure does not leak through the prompt. It is not, however, relied on as a control. Section 5 assumes the model may still name a restricted column, by guessing, by reusing an earlier answer, or by being prompted to, and blocks it regardless.

4.2 Selection: lexical scoring with relationship expansion

For each question the catalog tokenises the text, splitting CamelCase identifiers and adding simple singular forms ("categories" → "category", "orders" → "order"). Generic words that say nothing about which table is wanted ("show", "details", "information", "all") are removed. Each table is then scored as follows:

Signal	Score	Example
Every part of the table name (or alias) matched	20 per part	"suppliers" → `suppliers` scores 20
Some parts of the table name matched	5 per part	"order" alone → `order_details` scores 5
A column-name (or alias) word matched	3 per word	"discontinued" → `products.discontinued`

Table 2. Table-relevance scoring in the schema catalog.

The top ten tables are selected and expanded by following foreign keys to a configurable depth (default one). Relationships matter: a question about orders and customers may need joins across several tables, and supplying the actual foreign keys is more reliable than leaving the model to infer joins from similar column names. When no table is matched in full, the match is treated as weak and a short, configured list of core fallback tables is added. When nothing matches at all ("show me Nancy's information" names a person, not a table), the fallback schema is used on its own. The fallback is built from real, security-filtered columns, not a hand-written guess.

The selected slice is formatted as plain text: each table with its columns, types and nullability, followed by a `RELATIONSHIPS` block listing the foreign keys between selected tables. Labels, aliases and term definitions from the semantic model (Section 3.3) are shown beside the names they describe.

4.3 Domain routing: Who, What and Where

Lexical scoring alone does not scale to an ERP, where "order" may match sales, purchase, production and return orders across hundreds of tables. The agent therefore first routes each question to a Domain of Discourse (DoD): a named area of the authorised schema, such as Sales or Human Resources, defined as a group of tables plus the words, roles and locations that point to it. Northwind has five. Each domain is scored on three signals (Table 3).

Signal	Scored on	Weight
What	The domain named in the question, plus how well its best tables match the question's words	0.45
Where	The screen or function the user is working in; codes such as PUR0410 map to names through alias metadata	0.35
Who	The strongest affinity between the user's roles and the domain	0.20

Table 3. Signals used to route a question to a Domain of Discourse.

An absent signal's weight is shared among the others, and the previous turn's domain gets a 0.10 bonus so follow-ups stay put. A domain scoring at least 0.25 and 0.15 ahead of the next is searched alone; if the top two are closer, both are searched; otherwise the whole schema is searched (Section 4.2).

Routing sets where the search starts, not where it stops: anchors are found in the domain, but expansion follows the foreign-key graph up to a “hop limit”, so cross-domain questions still work. It is deterministic and changes context, never access. Domains are built from the security-filtered catalog, the LLM never chooses the domain.

4.4 Conversational context

Follow-up questions often lack enough information on their own. In "Which customer ordered it?" there is no product, order or customer table named. The agent therefore keeps the last five turns, that is, the question, the generated SQL, and the answer, and includes them in the generation prompt as background. The prompt tells the model to use this context only to resolve references such as "it", "them" or "those customers", and never to copy earlier results into SQL. Because earlier SQL already names its tables and joins, the model can carry an established entity (a product, then its order, then its customer) forward from turn to turn.

Conversation also shapes retrieval. The previous turn's domain receives a bonus, and a follow-up with no strong signal of its own (one that names no table, domain or location, such as "What

is Jose's phone number?") stays in that domain and starts from the tables the previous query used (Section 4.3).

5. Agent Behavioural Controls (ABCs)

The ABCs are the deterministic controls that constrain what the agent can do, regardless of what the LLM generates. There are four: SQL validation and authorisation, SQL rewriting and safeguards, controlled execution, and result security. They are supported by schema grounding (Section 4) and by observability (Section 5.5). The public demonstration site presents the same controls as six finer-grained boundaries, listing authorisation separately from validation and query constraints separately from rewriting.

Two design rules run through all four. First, **structure, not strings**: generated SQL is parsed into an abstract syntax tree with sqlglot and inspected node by node, never searched for dangerous keywords. Second, **visibility is not security**: the command-line agent shows the generated SQL so the user can see what the agent is attempting, but nothing is allowed simply because it looks reasonable.

5.1 SQL validation and authorisation

Validation runs in three stages, each returning a specific internal reason on failure.

- **Single statement.** The text must parse into exactly one statement. Stacked queries (`SELECT 1; DROP TABLE orders`) are rejected.
- **Structural safety.** The root must be a `SELECT` or a `UNION`. The whole tree is then searched for forbidden nodes: `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `DDL`, `COPY`, raw commands and row locks (`FOR UPDATE`, `FOR SHARE`). This matters because PostgreSQL executes data-modifying CTEs even when the outer statement is a `SELECT`: `WITH x AS (DELETE FROM orders RETURNING 1) SELECT 1` parses as a plain select. `SELECT INTO` is rejected because it creates a table.
- **Authorisation.** Every table node, including those in subqueries and joins, must be in the caller's authorised set; aliases are mapped back to physical tables. Every column node is checked against the denied list for its table. An unqualified column is checked conservatively against every referenced table. Whole-row references, which would return every column of a table as one value and bypass per-column checks (`SELECT e FROM employees e`, `row_to_json(e)`, `to_jsonb(e.*)`), are rejected.

Because authorisation works on the parsed query, it sees the columns actually referenced, wherever they appear. A restricted column used in a `WHERE`, `JOIN`, `ORDER BY` or inside an expression such as `CONCAT(phone, fax)` is blocked, since filtering or deriving from a value can reveal it.

5.2 SQL rewriting and safeguards

Authorisation decides whether a query may run; rewriting constrains how it runs. Even a permitted query can return too many rows or run too long.

- **Column withholding.** If a restricted column appears only as a plain output in the `SELECT` list, it is removed and the rest of the query proceeds. The answer carries a generic "fields withheld" note. Restricted columns used anywhere else are left in place so that validation blocks them.
- **Wildcard expansion.** `SELECT *` and `alias.*` are rewritten into the explicit list of authorised, non-denied columns for each referenced table.
- **Row limit.** If the query has no `LIMIT`, one of 1,000 rows is added. The limit is applied by code, not by hoping the model remembers to include it.
- **Re-validation.** The rewritten SQL is validated again from scratch, so a rewriting fault cannot produce an unchecked query.

When a result may be truncated, the system should not imply it considered every matching record. The artificial limit is removed from the SQL shown to the explanation model, so it is not mistaken for part of the user's request.

5.3 Controlled execution

The model never holds a connection or credential. SQL reaches the database only through one function, which applies three independent boundaries:

1. a final structural-safety check immediately before execution;
2. a read-only transaction, enforced by PostgreSQL itself;
3. a statement timeout (10 seconds by default) scoped to that transaction.

The connection uses a dedicated database role with `SELECT` grants only on the authorised tables. This means that even a complete failure of the application-level checks would still meet a database that refuses writes and access to other tables.

5.4 Result security and response

Rows are checked again on the way out, before they reach the model or the user. Any result key matching a denied column name is removed, and results are capped at 1,000 rows and 5 MB. This is defence in depth; primary authorisation has already happened before execution.

Refusals are designed not to leak. Each block maps to a stable rule code with a short label and a generic message that never names a table or column (see Table 4),

The model's own markers are handled conservatively. A "write request" marker is only reported as a read-only block when the question contains a write-style verb; otherwise it is treated as data not available. The keyword check only chooses which message to show.

Rule	Label shown	Triggered by
`restricted_data`	restricted data	A denied table or column in the query, or a question about a restricted field the model reported as unavailable
`not_available`	not available	A data question the authorised schema cannot answer
`read_only`	read-only	Any write, DDL, `SELECT INTO` or row lock; or a request to modify data
`single_statement`	one query only	More than one statement
`unparseable`	not validated	SQL the parser cannot read (reported as an error, not a policy block)
`policy`	security policy	Any other validation failure
`fields_withheld`	fields withheld	Restricted columns removed from an otherwise answered query

Table 4. Block rules. The detailed reason stays in the server console and audit log.

5.5 Observability

Each step is logged: retrieved schema size, generated SQL, SQL after withholding and rewriting, security decisions with their detailed reasons, row counts, corrections and the final answer. The public API adds an append-only audit log with one line per event — request received or completed, rate-limited, busy, timeout, error — recording a request ID, a hashed client IP, the session ID, the question and the executed SQL. The command-line agent shows the user the generated SQL. The public demo is more conservative: it logs the SQL but returns only the answer and the rule label to the browser, and never the detailed block reason.

6. Mapping to IBAAM

Secure SQL Agent implements a subset of IBAAM's principles in a single domain: the invariants sit above the planner, the adaptive parts are free to vary, and the ABCs are the control layer that keeps one from affecting the other. IBAAM is a much broader framework; the ABCs are a practical implementation of part of it (see Table 5).

In IBAAM's formal terms, an agent is the set of states satisfying every invariant, and a transition is valid only if the invariants hold before and after it (Long, 2026a). Here each question is one transition: the proposed SQL is the candidate action, and the ABCs check it against the invariants before execution (validation, re-validation, the execution gate) and after execution (the result policy). A proposal that fails any check is rejected, and the system's state is unchanged apart from the log.

IBAAM element	In Secure SQL Agent	Who decides
Invariants	Read-only access; authorised tables and columns only; one statement per request; bounded execution (rows, bytes, time)	Fixed by the security policy and configuration
Adaptive layer	Schema selection per question; conversational interpretation over five turns; SQL proposal; one-shot correction; result explanation	The LLM and retrieval heuristics, free to vary
Control layer	The ABCs: AST validation and authorisation, column withholding, wildcard rewriting, row limits, re-validation, result policy	Deterministic code
Execution layer	One execution function: final safety check, read-only transaction, statement timeout, SELECT-only role	Deterministic code and the database
Observability layer	Console trace of every step, generated and rewritten SQL, authorisation decisions, errors, results; audit log with request IDs	Deterministic code

Table 5. Secure SQL Agent mapped to IBAAM.

Three properties follow from this placement.

1. **The model cannot widen its own authority.** Nothing the LLM emits is read as a permission. Its markers can only produce a refusal; its SQL is only ever narrowed (columns withheld, limits added) or rejected.
2. **Controls do not depend on model behaviour.** Every boundary runs whether the request is routine or adversarial, and whether the model follows its prompt or not. A prompt injection that persuades the model to write `DELETE` produces a `DELETE` that the parser rejects.
3. **Defence in depth is layered by mechanism, not repeated by copy.** Schema filtering, AST authorisation, rewriting, the execution gate, the database role and the result policy use different mechanisms, so a single bug is unlikely to defeat all of them.

As the series progresses, the later layers become relevant. Agent Identity Evals could measure drift in the agent's SQL-generation behaviour across model upgrades (Long, 2026b). HIG and HIG-C would matter once invariants can conflict, for instance a duty to answer completely against a row limit (Long, 2026c; 2026d). The conscience layer proposed in the fifth paper would sit between plan validation and execution (Long, 2026f), which is exactly where Secure SQL Agent's validation sits; any such layer could only add grounds for refusal, never authorise a query the ABCs block.

7. Demonstration

The agent can follow a chain of related questions to the right record, yet still refuses the one field the user is not authorised to see. The demonstration uses Northwind, a sample database modelling a food distributor. The demo role in the security policy authorises eight tables (customers, orders, order details, products, categories, suppliers, employees, shippers) and denies four columns: customer phone, fax and address, and employee home phone.

7.1 A conversational walkthrough

Turn	User	Agent
1	Show me all the discontinued products.	Lists 8 discontinued products, including Perth Pasties.
2	When was Perth Pasties last ordered?	Perth Pasties was last ordered on 1 May 1998.
3	Which customer ordered it?	Save-a-lot Markets (contact: Jose Pavarotti).
4	What is Jose's phone number?	That information is restricted, so this question can't be answered.

Table 6. A four-turn conversation with the agent.

Turns 2 to 4 depend on conversation context. Turn 2 names a product but not an order. Turn 3 names neither, so "it" must be resolved to Perth Pasties and its most recent order from the previous turn's question, SQL and answer. Turn 4 resolves "Jose" to the contact of the customer found in turn 3.

The refusal in turn 4 is not the model declining politely. It is the controls. There are two routes, and both end in the same block:

- **The model never sees the column.** `customers.phone` is filtered from the catalog, so the schema contains no customer phone. The model replies with the data-not-available marker, and because the question mentions a denied field ("phone"), the agent reports it as `restricted_data`.
- **The model writes it anyway.** If the model guesses `SELECT phone FROM customers WHERE contact_name = 'Jose Pavarotti'`, withholding would leave an empty `SELECT` list, so the query is left intact and AST authorisation rejects `public.customers.phone` as not authorised. The query never reaches the database.

Either way, the agent has understood exactly what is being asked and has identified the data. It cannot override the access control.

7.2 Write requests

A plain LLM agent asked to "clean up test accounts" might produce `DELETE FROM customers WHERE email LIKE '%test%'` and run it. Secure SQL Agent's generation prompt asks the model to return a write-request marker instead, and the agent reports a `read-only` block. If the model ignores the prompt and writes the `DELETE`, the parser finds a non-`SELECT` root and rejects it. If it hides the delete in a CTE, the forbidden-node search finds it. If every application check failed, the read-only transaction and the SELECT-only role would still refuse it.

7.3 A permitted analytical query

For "Which products generate the most revenue?" the server log records 60 lines of retrieved schema, then this generated SQL:

```
SELECT p.product_name, SUM(od.quantity * od.unit_price * (1 - od.discount))
AS total_revenue
FROM products AS p
JOIN order_details AS od ON p.product_id = od.product_id
GROUP BY p.product_name
ORDER BY total_revenue DESC
LIMIT 10
```

The query passes validation. Because it already has a `LIMIT`, rewriting leaves it unchanged apart from normalised formatting. Ten rows are returned and the model explains them, naming Côte de Blaye as the highest-revenue product at $141,396.74, followed by Thüringer Rostbratwurst and Raclette Courdavault. The model has correctly joined two tables using the foreign key supplied in the schema slice and applied the discount factor.

8. Discussion and limitations

Secure SQL Agent is a working prototype that brings together the architectural patterns needed for a secure data agent, but it is not a production system and has not been evaluated quantitatively. This section states what it does and does not guarantee.

8.1 What the controls guarantee

Under the assumption that the security context is correct and the parser models the target SQL dialect faithfully, the controls ensure that no statement other than a single read-only query reaches the database, that no authorised-table check is skipped, and that no explicitly denied column is referenced, returned or passed to the model. These guarantees hold regardless of the model's output, which is what distinguishes them from prompt-level rules. Their strength rests on established, inspectable mechanisms (parsing, allow-lists and database privileges) rather than on model behaviour.

8.2 What they do not guarantee

- **Answer accuracy.** The controls decide what may run, not whether the SQL answers the question correctly. A permitted but wrong join returns a confident, wrong answer. Showing the SQL helps a technical user spot this; it does not solve it.
- **Inference from permitted data.** Column-level controls cannot stop a user from inferring restricted facts from permitted ones, for example through aggregates over small groups. Row-level security and inference control are not yet implemented.
- **Faithful explanation.** The explanation model is told not to invent information, but its summary is not checked against the rows.
- **Lexical retrieval.** Table selection is keyword-based. Synonyms, jargon and cryptic names not listed in the semantic model will be missed, at which point the fallback tables carry the question. Follow-ups that name no table, domain or location reuse the previous query's tables; other questions are matched on their own words alone.
- **Parser dependence.** Security depends on sqlglot interpreting a statement the way PostgreSQL does. Divergence between parser and engine is a classic source of bypasses, which is why the database-level read-only transaction and SELECT-only role remain essential.
- **Identity in the demo.** The public API has no user authentication. All anonymous visitors share one fixed, narrow security context; session IDs separate conversation history and rate-limit buckets but do not prove identity.

8.3 Operational controls

Enterprise environments impose practical constraints beyond authorisation: large schemas, automated discovery, query and execution limits, connection timeouts and shared resources. The demo addresses several of these at the API boundary. Questions are capped at 300 characters. Requests are rate-limited per IP (10 a minute, 150 a day) and per session (4 a minute, 40 a day). At most two agent runs hold model capacity at once, with a five-second queue wait. Each request has an overall timeout, and a stuck run can hold a slot only for a bounded time. The database statement timeout is kept well below the request timeout, so queries abandoned by a timed-out request cannot tie up connections for long.

8.4 Path to enterprise grade

Moving from prototype to enterprise deployment mainly means extending and hardening the existing controls rather than changing the architecture:

- more comprehensive SQL validation and rewriting, including function allow-lists and cost-based limits from the query planner;
- row-level and value-level security, masking and sensitive-data classification in the result policy;

- end-to-end integration with enterprise identity: the security policy already maps identity-provider groups to roles, but the demo has no sign-in and roles are not yet mirrored in database roles;
- a fuller conceptual model for each domain, built on the semantic model's labels, relationships and terms, with semantic or embedding-based retrieval alongside alias matching;
- additional resource controls, connection pooling and multi-process rate limiting;
- systematic evaluation: answer accuracy against a benchmark, and red-team testing of the controls with adversarial prompts and SQL.

9. Conclusion

Generating SQL and returning an answer is relatively straightforward; building an agent that operates inside defined enterprise constraints is not. Secure SQL Agent shows one way to do it. Schema discovery and selection give the model focused, relationship-aware grounding without exposing the whole database. Conversational context lets users ask follow-up questions naturally. Deterministic authorisation, AST-based validation, rewriting, controlled execution and result filtering (i.e. the Agent Behavioural Controls) decide what actually happens. None of these solves every problem alone. Together they produce an agent whose behaviour is governed by controls outside the LLM.

The broader point is architectural. The model interprets intent; deterministic code enforces the invariants; and the invariants sit above the planner and tool-execution layers where the model cannot reach them. That is IBAAM's central principle, and Secure SQL Agent is a concrete demonstration that safe, secure, behaviour-bound agents can be built this way in practice, rather than relying on prompts or model behaviour to provide those guarantees. Before an agent is released into the world, it should respect its ABCs.

Use of generative AI. Generative AI tools were used to assist with drafting and editing this manuscript. The system, its design and the underlying ideas originated with the author, who reviewed the content and is responsible for it.

References

Li, J., Hui, B., Qu, G., et al. (2023). Can LLM Already Serve as a Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *NeurIPS 2023 Datasets and Benchmarks*. arXiv:2305.03111

Long, B. (2026a). Engineering Stable Behaviour in Self-Modifying LLM Agents. *Adaptive Agent Governance Research Series No. 1*. April 2026. Available at SSRN. doi:10.2139/ssrn.6888518

Long, B. (2026b). Measuring Behavioural Stability in Adaptive Agent Systems. *Adaptive Agent Governance Research Series No. 2*. May 2026. Available at SSRN. doi:10.2139/ssrn.6888539

Long, B. (2026c). Hierarchical Invariant Governance for Adaptive Agents. *Adaptive Agent Governance Research Series No. 3*. June 2026. Available at SSRN. doi:10.2139/ssrn.6888638

Long, B. (2026d). Context-Aware Hierarchical Governance for Adaptive Agents. *Adaptive Agent Governance Research Series No. 4*. June 2026. Available at SSRN. doi:10.2139/ssrn.7006318

Long, B. (2026e). InvariantAgent2 (IA2): Reference Implementation of the Invariant-Bounded Agent Alignment Model. GitHub. github.com/xagyg/InvariantAgent2

Long, B. (2026f). From Invariants to Conscience: Toward Internalised Judgment in the Governance of Adaptive Agents. *Adaptive Agent Governance Research Series No. 5*. August 2026. Available at SSRN. doi:10.2139/ssrn.7220559

Long, B. (2026g). Secure SQL Agent: live demonstration. sqlagent.net

Perrier, E., & Bennett, M. T. (2026). Agent Identity Evals: Measuring Agentic Identity. Preprint. doi:10.48550/arXiv.2507.17257

Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS 2023*. arXiv:2302.04761

Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. arXiv:2210.03629

Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP 2018*. arXiv:1809.08887